

Sprevonix Consult

Lab Documentation Cloud & DevOps Capstone Project 2

Student Name	Chizoba Onwuka
Course	Cloud & DevOps
Project	Capstone Project — Sprevonix Consult
Date Submitted	2026-08-31
Instructor	Dr Agboola
Platforms	Amazon Web Services (AWS), GitHub

Table of Contents

Table of contents

Part 1 Local Development & GitHub Repository

Description.....	3
Preparation.....	3
Observations.....	3
Screenshots.....	4
Reflection.....	5

Part 2 Static Website Hosting on Amazon S3

Description.....	5
Preparation.....	5
Screenshots.....	6
Reflection.....	9

Part 3 HTTPS Delivery with Amazon CloudFront

Description.....	9
Preparation.....	9
Observations.....	11
Screenshots.....	12
Reflection.....	14

Part 4 Creation of the AWS Code pipeline

Description	14
Preparation.....	15
Reflection	20

Part 5 Custom Domain Registration using Amazon Route 53

Description	22
-------------------	----

References	24
------------------	----

Part 1 — Local Development & GitHub Repository

Description

The objective of Part 1 is to build the Sprevonix consulting website locally and upload all source files to a public GitHub repository. The platform is a static website composed of five files: about.html, contact.html, index.html, style.css, and main.js. The website enables users to view the home page of Sprevonix consulting and look at what they do and the services they offer.

Preparation

The following preparatory steps were completed to ensure source files are uploaded to GitHub repository.

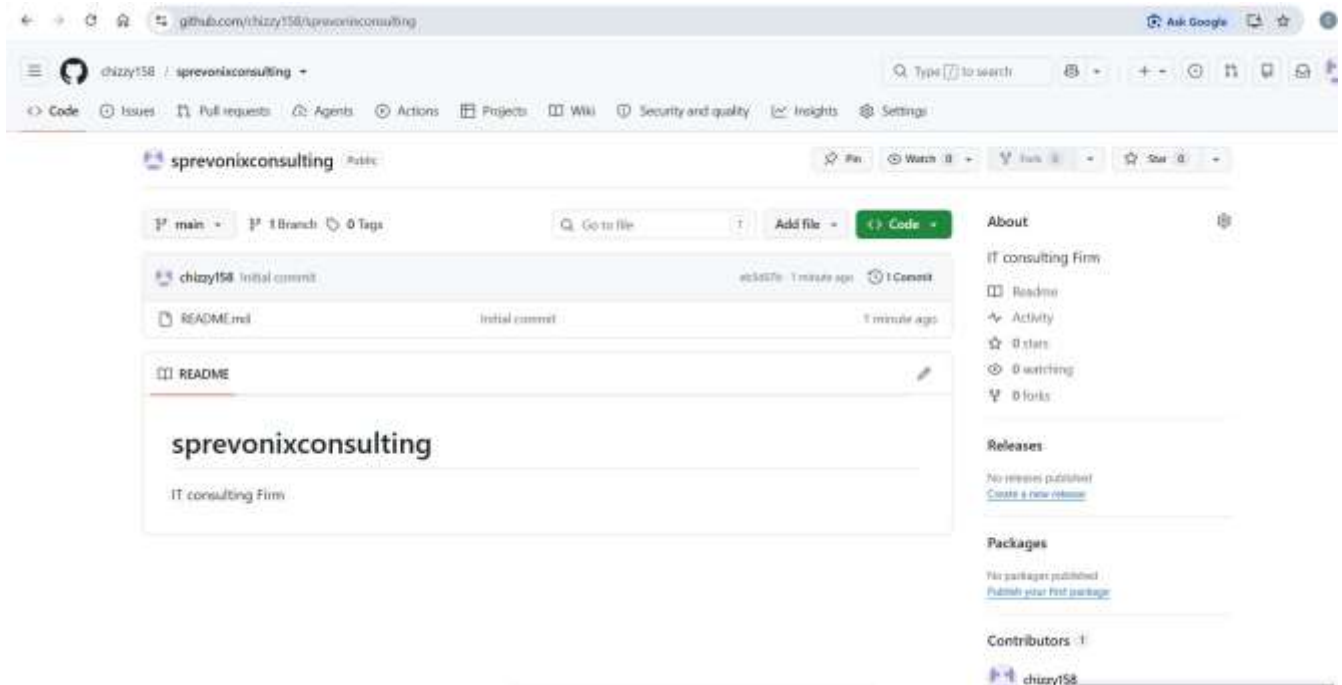
First a project folder named sprevonixconsulting was created on the local machine: through the following steps. Github was opened and from the GitHub dashboard, click “+” icon located at the top-right corner of the page was clicked and “New repository” was selected. This opens the repository creation page. Then sprevonixconsulting is added as the name of the repository as shown in screenshot 1 Then Public is selected to make the repository to be visible to everyone. All five website files (about.html, contact.html, index.html, style.css, main.js) were successfully uploaded to the GitHub repository as seen in screenshot 2. Additionally, a README.md file was included to document the project architecture, deployment procedures, and recommendations.

Observations

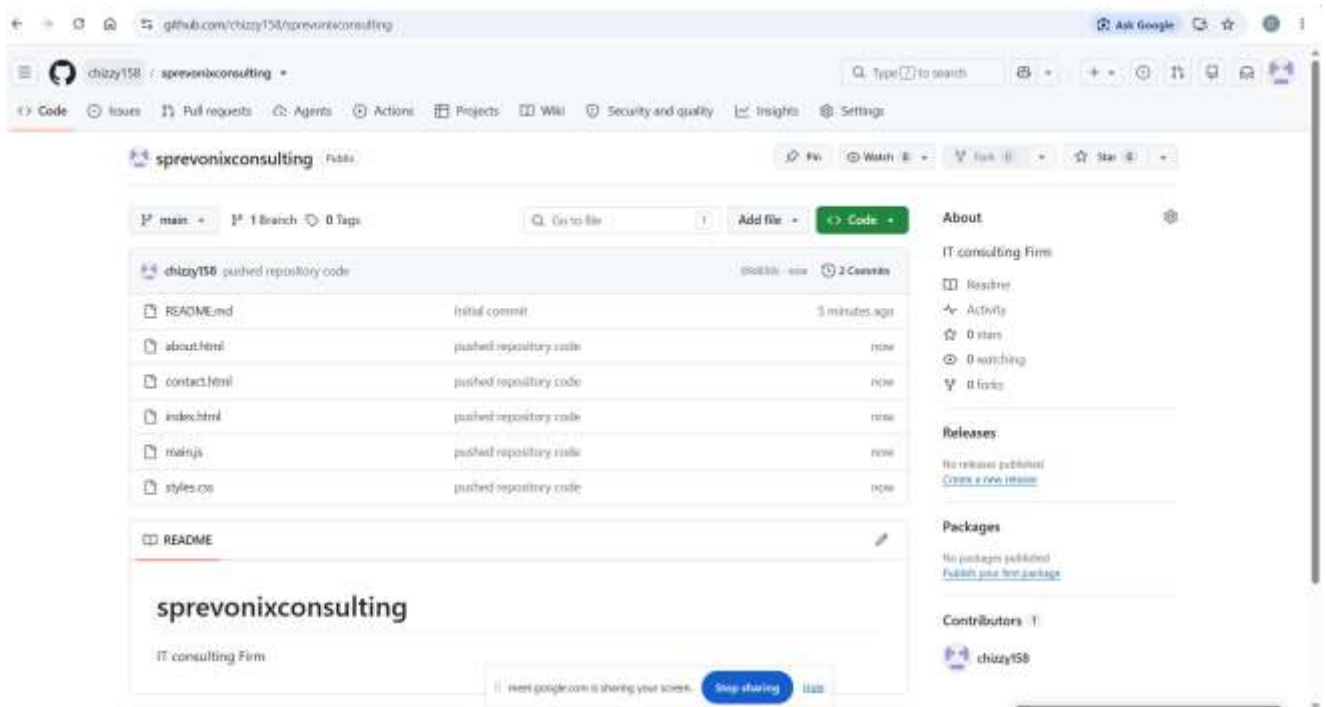
The deployment completed successfully and the website was accessible through the generated endpoints.

Screenshots

Screenshot 1 Creating Github Repository for this project.



Screenshot 2 — Files uploaded-Browser showing index.html, main.js and style.css already uploaded in repository file folder



Reflection

The initial phase of the project established the foundational architecture for the entire system. Maintaining the application as a static website, consisting of only five files and lacking server-side dependencies, ensures compatibility with Amazon S3 hosting. For startups such as Sprevonix Consulting, presenting a clean, organized, and well-documented GitHub repository to potential investors signals robust engineering practices and a commitment to professional development standards.

Part 2 — Static Website Hosting on Amazon S3

Description

The objective of Part 2 is to host the Sprevonix consulting static website on Amazon S3. Amazon S3 offers highly available, durable, and cost-efficient object storage services. By enabling the Static Website Hosting feature, S3 can deliver HTML, CSS, and JavaScript files directly to users over the internet without the need for server administration or maintenance.

Preparation

The AWS Management Console was accessed with account credentials. In the search bar, S3 – Scalable Storage in the Cloud was selected to open the Amazon S3 dashboard. On the dashboard, the create bucket option was chosen. The bucket name 'sprevonixconsult' was entered as seen in screenshot 3 and the AWS Region was set to US East (N. Virginia) – us-east-1. Under Object ownership, the recommended option to disable ACLs was retained before creating the bucket.

The newly created 'sprevonixconsult' bucket was opened, and the required files were uploaded to Amazon S3, as showed in screenshot 4.

A bucket policy was added to grant the s3:GetObject permission for the website objects as shown in screenshot 5.

The Host a static website option was selected, and 'index.html' was entered in the Index document field.

The Permissions tab of the S3 bucket was accessed. For direct S3 website hosting, the bucket was configured to allow public read access to the website files.

The properties tab of the S3 bucket was accessed to locate the static website hosting section, where it was enabled as seen in screenshot 6 and thereafter the link bucket website endpoint was opened as seen in screenshot 7.

Screenshots

Screenshot 3-S3 Bucket Created

us-east-1:console.aws.amazon.com/s3/bucket/create?region=us-east-1

Amazon S3 > Buckets > Create bucket

Create bucket

Buckets are containers for data stored in S3.

General configuration

AWS Region
US East (N. Virginia) us-east-1

Bucket type

- ☒ **General purpose**
Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.
- ☐ **Directory**
Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.

Bucket namespace
Choose the namespace where you want to create your bucket. [Learn more](#)

- ☒ **Global namespace**
By default, S3 creates general purpose buckets in the global namespace.
- ☐ **Account Regional namespace (recommended)**
General purpose buckets created in your account Regional namespace are unique to your account. These buckets can never be created by another AWS account.

Bucket name

Spirevix Consulting Service

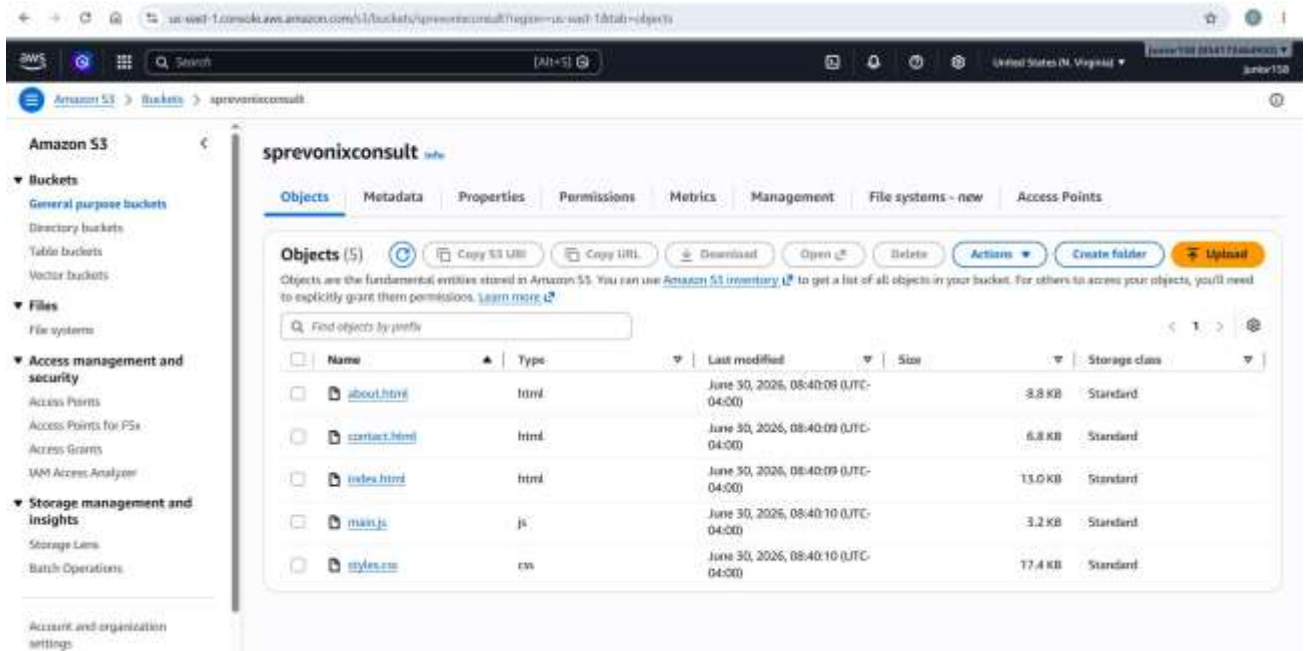
Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn more](#)

Copy settings from existing bucket - optional
Only the bucket settings in the following configuration are copied.

[Choose bucket](#)

Format: s3://bucket/prefix

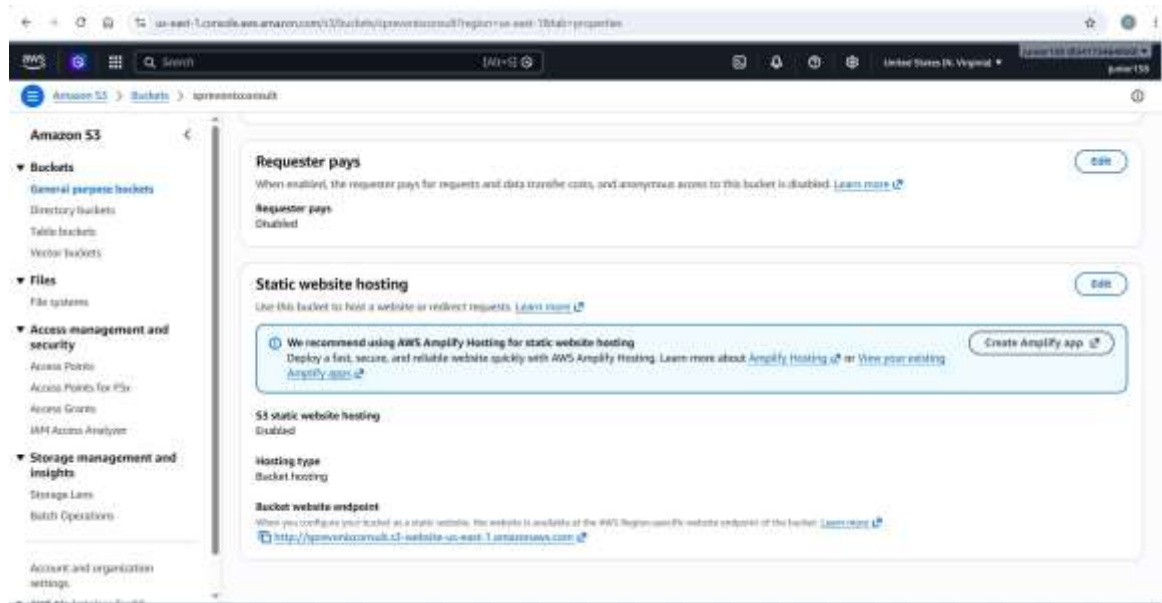
Screenshot 4-Files Uploaded to S3(S3 bucket root showing index.html, style.css, main.js, etc. uploaded)



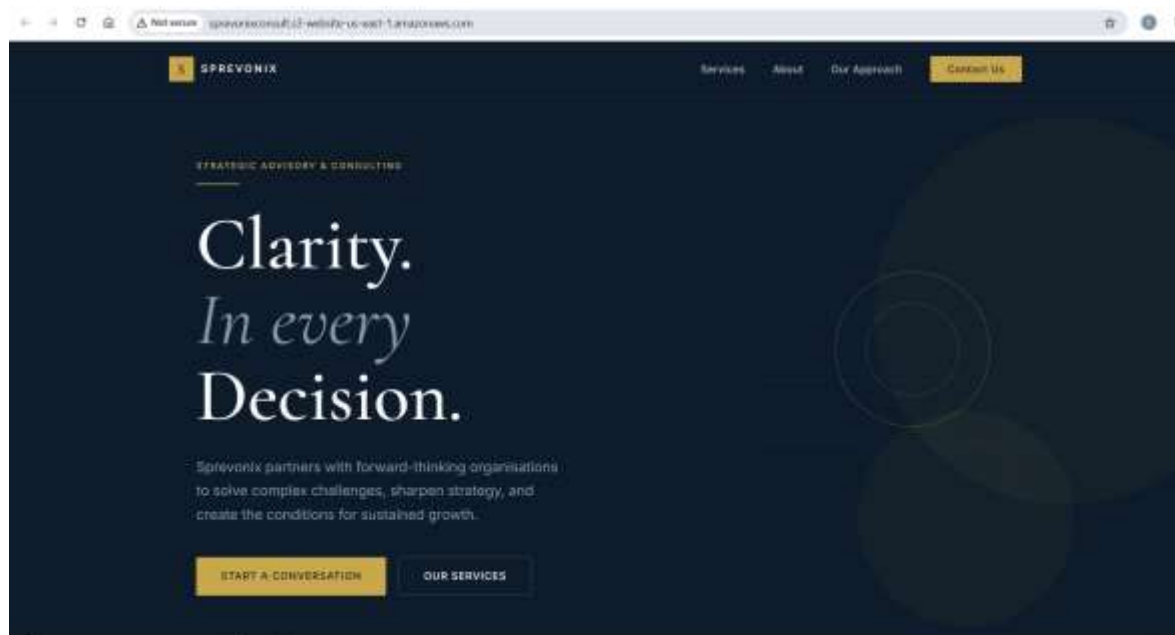
Screenshot 5 —



Screenshot 6 — Static Website Hosting Enabled and S3 static endpoint website was tested.



Screenshot 7 — Static Website Hosting Enabled and S3 static endpoint website was tested.



Reflection

Deploying a static website on Amazon S3 represents a highly cost-effective strategy within cloud engineering. Given that SprevonixConsult is a fully static application with no backend processing or database queries during page loads, Amazon S3 is an optimal hosting platform.

A critical consideration during deployment is the requirement to upload all website files directly to the root directory of the S3 bucket. Placing files within a subfolder can prevent the S3 static website hosting service from locating the index.html file, resulting in a 404 error. To ensure proper website loading, all necessary files should be present in the bucket's objects tab, not nested in subdirectories.

Initially, the website was accessible exclusively via the S3 endpoint over unencrypted HTTP. Because HTTP does not provide data encryption, it is unsuitable for production environments. This limitation was addressed in part 3 through integrating Amazon CloudFront with S3, which enabled secure HTTPS access.

Part 3 — HTTPS Delivery with Amazon CloudFront

Description

The purpose of this section is to configure Amazon CloudFront in front of the Amazon S3 bucket so that the Sprevonix website can be delivered securely over HTTPS. CloudFront also improves the performance of the website by delivering content through AWS edge locations around the world.

The services utilized in this phase include:

- Amazon CloudFront CDN – Delivers website content globally while enforcing HTTPS connections.
- Origin Access Control (OAC) – Prevents direct access to the S3 bucket by allowing only CloudFront to serve the website files.
- Default CloudFront SSL Certificate – Provides free HTTPS support through the cloudfront.net subdomain.

Preparation

The AWS Management Console is accessed, and CloudFront is entered into the AWS search bar. CloudFront is then selected from the available AWS services. On the CloudFront dashboard, create distribution is selected to begin configuring a new

CloudFront distribution. Under the Origin section, the S3 bucket created in Part 2 is selected from the origin domain dropdown menu as shown in screenshot 8. For this project, the sprevoconsult S3 bucket is selected as the origin. This allows CloudFront to retrieve the website files stored in the S3 bucket and deliver them to users.

Next, Origin Access Control (OAC) is configured for the S3 origin. If an OAC has not already been created, a new one is created using the recommended settings. OAC allows CloudFront to access the files stored in the S3 bucket while preventing users from directly accessing the bucket. This provides an additional layer of security by ensuring that website content is delivered through CloudFront.

For the SSL/TLS certificate configuration, the Default CloudFront Certificate (*.cloudfront.net) is selected. This certificate allows the website to be securely accessed using the CloudFront domain name without requiring a custom domain or a separate SSL certificate from AWS Certificate Manager. The Default root object is then configured by entering index.html. This ensures that the website's homepage is automatically displayed when users enter the CloudFront domain name without specifying a particular webpage or file.

After the required settings have been reviewed, create distribution is selected. The CloudFront distribution is then created and deployed across the AWS network. As seen in screenshot 9.

Once Origin Access Control has been configured, the S3 bucket policy is updated to give CloudFront permission to retrieve the website files. The required OAC bucket policy is copied, and the Amazon S3 Console is reopened. The sprevoconsult bucket is selected, followed by the Permissions tab. Under Bucket policy, Edit is selected, and the previous public-access policy is replaced with the policy required for CloudFront OAC. The updated policy is then saved.

After the distribution has been deployed, the CloudFront Console is reopened, and the newly created distribution is selected. The Distribution domain name is located and copied as shown in screenshot 10.

To confirm that the configuration is working correctly, the CloudFront distribution domain name is opened in a web browser using HTTPS as seen in screenshot 11. The website is checked to ensure that it loads successfully, and the browser's secure connection indicator is verified to confirm that the connection is encrypted.

Observations

The CloudFront distribution was configured through the AWS Management Console using the following settings:

- Origin domain: the sprevonixconsult S3 bucket selected from the dropdown menu.
- Viewer Protocol Policy: Redirect HTTP to HTTPS.
- Default root object: index.html.
- SSL Certificate: Default CloudFront Certificate (*.cloudfront.net).

After selecting Create Distribution, CloudFront automatically generated an updated bucket policy. This policy was copied and pasted into the Permissions tab of the S3 bucket, replacing the previously configured public access policy.

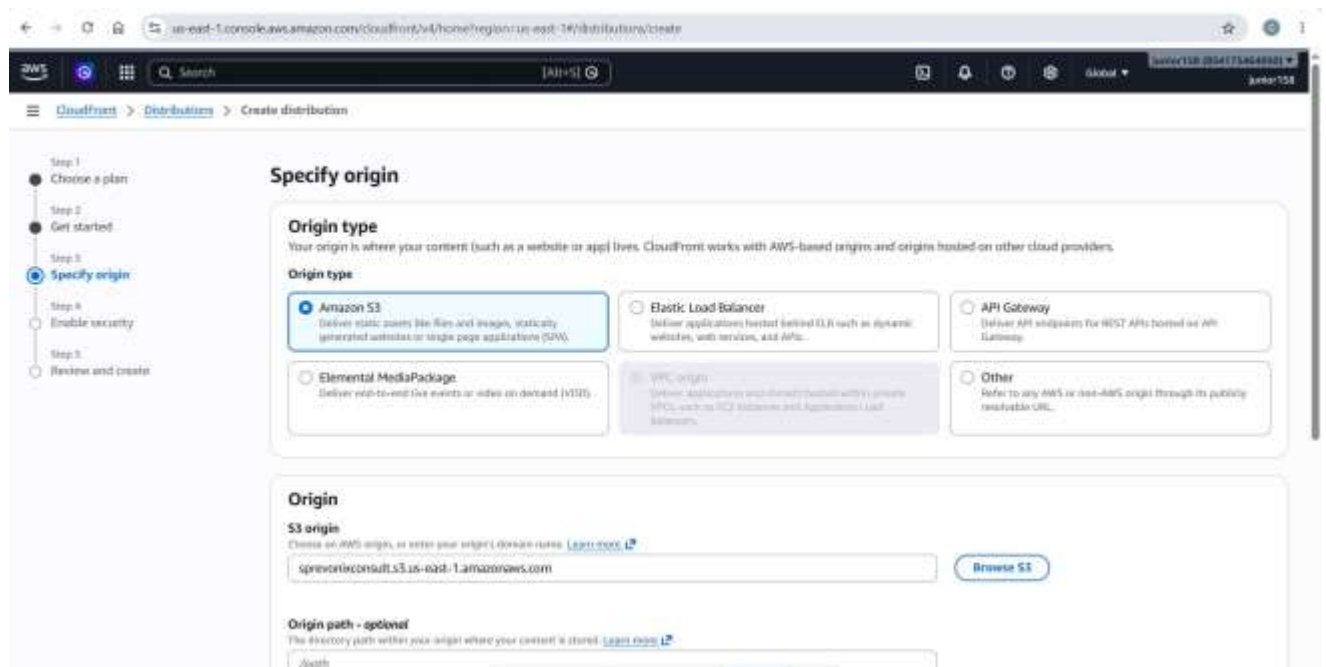
With Origin Access Control (OAC) enabled, the S3 bucket is no longer directly accessible. Instead, all requests must pass through CloudFront.

The distribution remained in the Deploying state for approximately 30 minutes before its status changed to Deployed. Once deployment was complete, the Distribution Domain Name was copied and used as the project's live HTTPS URL.

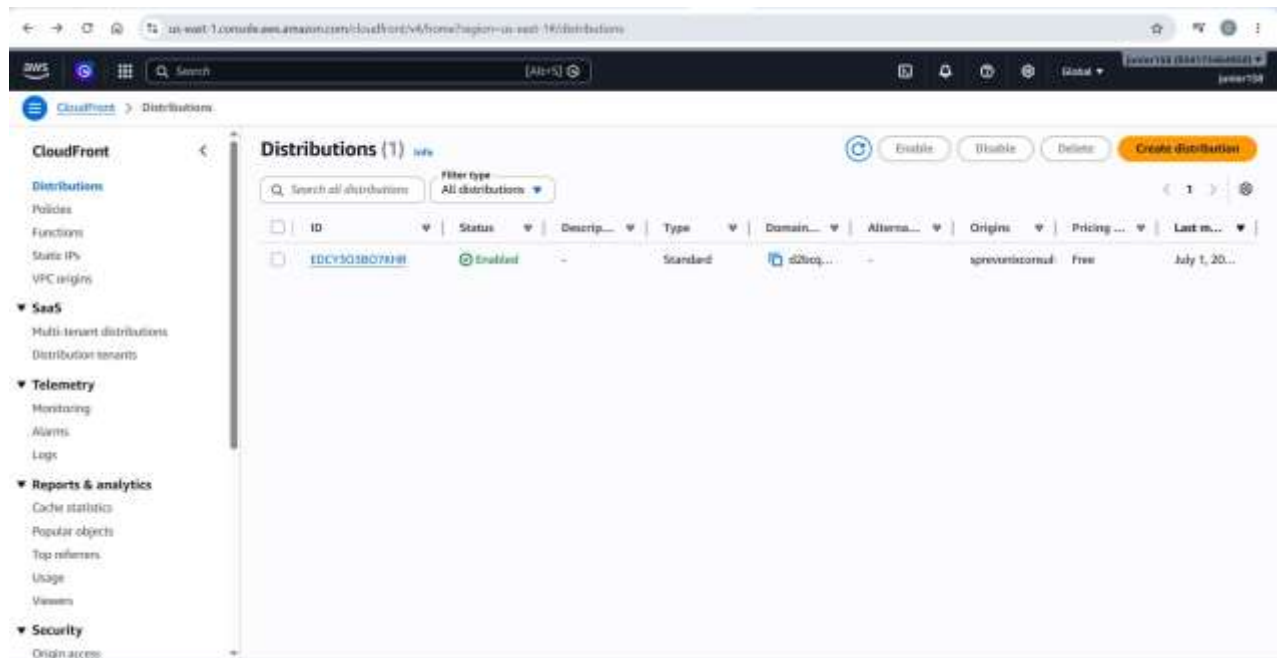
After the configuration has been completed and tested, Amazon CloudFront is able to deliver the Sprevonix website securely over HTTPS. The default CloudFront SSL/TLS certificate provides encrypted communication, while Origin Access Control prevents direct public access to the S3 bucket. As a result, website traffic is securely routed through CloudFront before the files stored in the private Amazon S3 bucket are delivered to users.

Screenshots

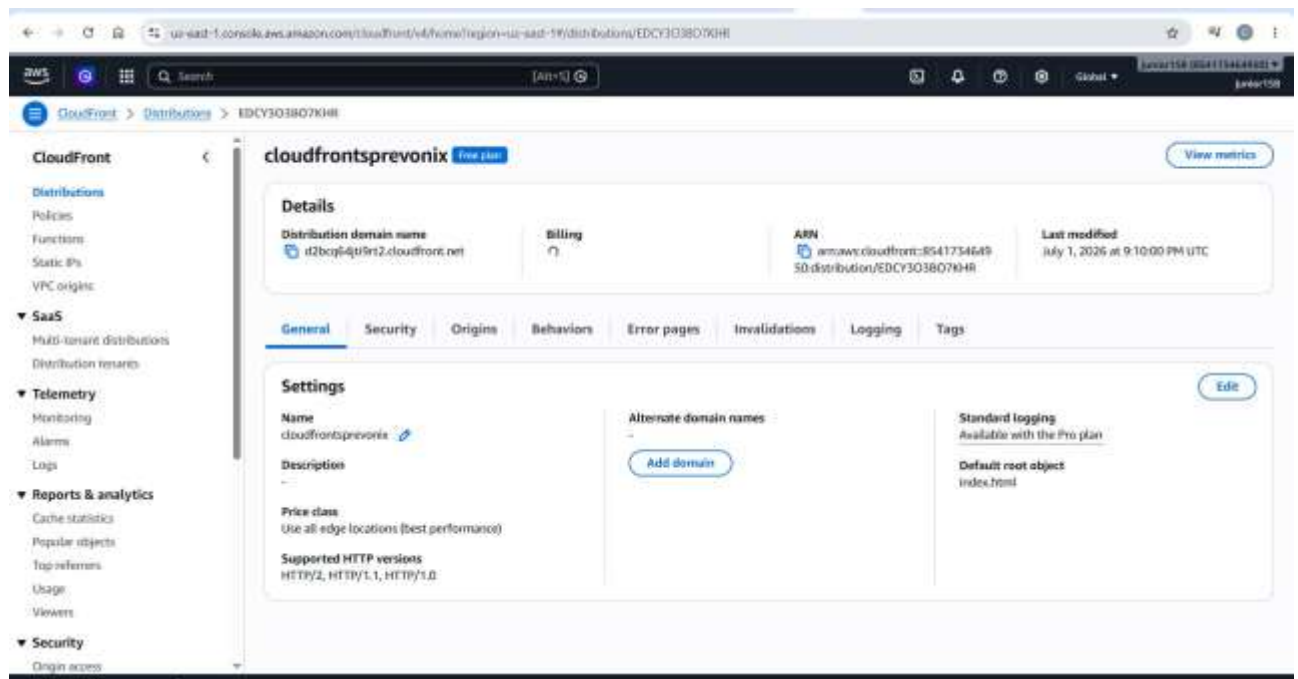
Screenshot 8— CloudFront Distribution Deployed



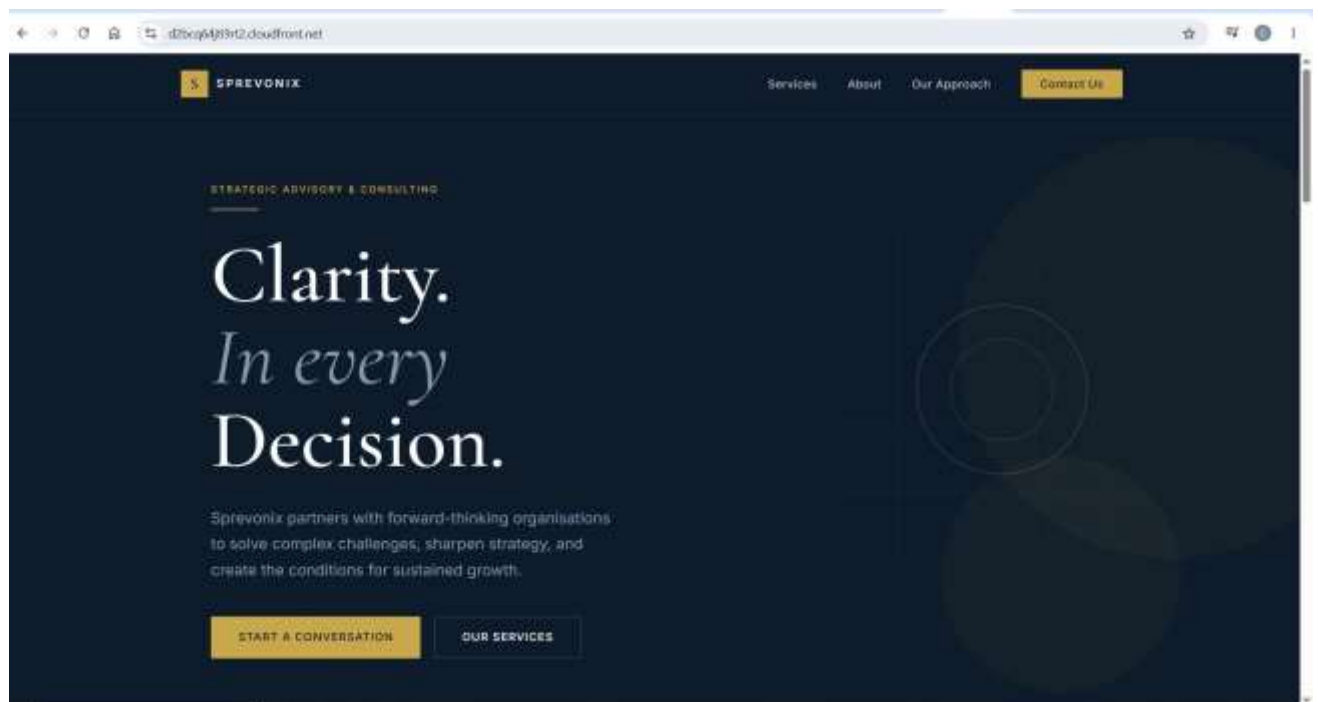
Screenshot 9— CloudFront Distribution created



Screenshot 10— CloudFront distribution domain name.



Screenshot 11 — Site Loading Over HTTPS



Reflection

Part 3 focused on the project's most important security objective: enforcing HTTPS across the platform. The default CloudFront certificate delivers the same end-to-end TLS encryption as a custom ACM certificate; the primary distinction is that the application uses a cloudfront.net URL instead of a custom domain. For a minimum viable product (MVP) intended for investor demonstrations, this approach is both practical and sufficient.

The implementation of Origin Access Control (OAC) significantly strengthened the platform's security posture. By enabling OAC, the S3 bucket is kept private and cannot be accessed directly, even if someone discovers the bucket's URL. Instead, all requests are routed through CloudFront, allowing future security controls—such as AWS WAF rules, geo-blocking, and rate limiting—to be applied at the content delivery network (CDN) layer.

In addition, the “Redirect HTTP to HTTPS” policy ensures that users who enter `http://` are automatically redirected to a secure connection without interruption. This configuration eliminates unsecured HTTP access, satisfies the project's security requirements, and provides users with a seamless browsing experience while protecting data in transit.

Part 4 — Creation of the AWS Code Pipeline

Description

In this section, AWS CodePipeline is set up to automate deploying the Sprevonix website from the GitHub repository to the Amazon S3 bucket. CodePipeline creates a continuous delivery workflow that detects changes in the source code and automatically moves them through each stage, and avoid deploying the updates manually.

The following services are used in this phase:

- AWS CodePipeline automates the continuous delivery process by moving website files from the source repository to the deployment environment.
- GitHub is the source repository where the Sprevonix website files are stored and updated.

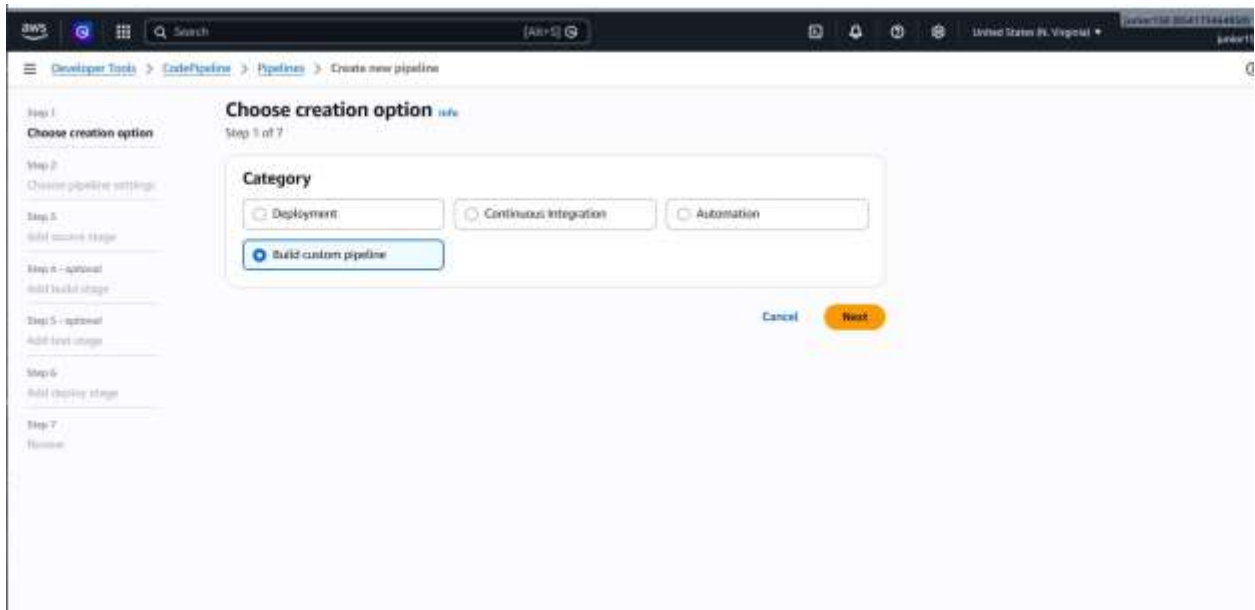
- GitHub Connection provides a secure link that lets AWS CodePipeline access the GitHub repository and detect any changes.
- Amazon S3 – Serves as the deployment destination where CodePipeline stores and hosts the website files.

Preparation

After GitHub and Amazon S3 have been prepared as shown in part 1 and 2 respectively, AWS Code Pipeline is configured to automate the deployment of the website files from GitHub to the S3 bucket. The process is as below,

The AWS Management Console is accessed, and CodePipeline is entered into the AWS search bar. AWS CodePipeline is then selected, followed by Pipelines. On the Pipelines page, create pipeline is selected, and build custom pipeline is chosen, as shown in screenshot 12.

Screenshot 12 — Creation of AWS Code pipeline.



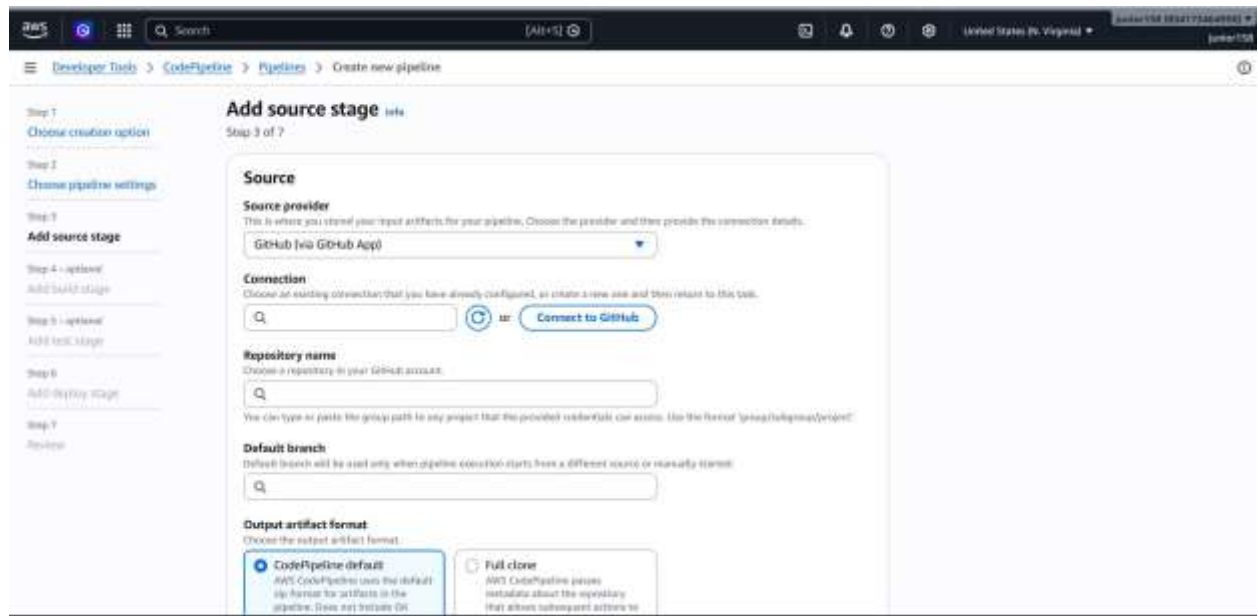
A pipeline name is then entered. For this project, sprevonixconsulting pipeline is used as the pipeline name. Under Service role, the option to create a new service role is

selected so that the required permissions can be provided to CodePipeline. Next is then selected to proceed.

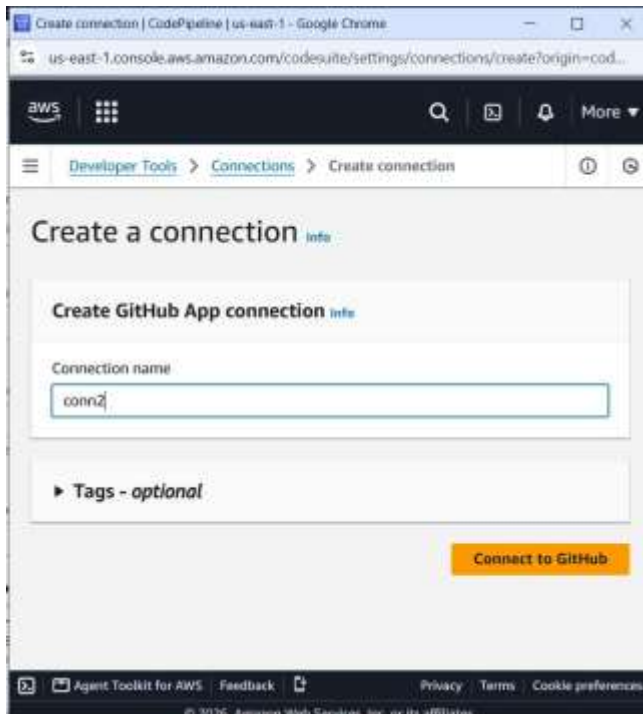
The GitHub repository is then connected to AWS CodePipeline through the Source stage. Under Source provider, GitHub (via GitHub App) is selected, as shown in screenshot 13. This connection mechanism is used by AWS to authorize CodePipeline to communicate securely with the GitHub repository.

Under Connection, connect to GitHub is selected. The GitHub authorization page is then displayed, as shown in screenshot 14 The connection name conn2 is entered, after which Connect to GitHub is selected to authorize the connection.

Screenshot 13 — GitHub repository connected to AWS CodePipeline through the Source stage.

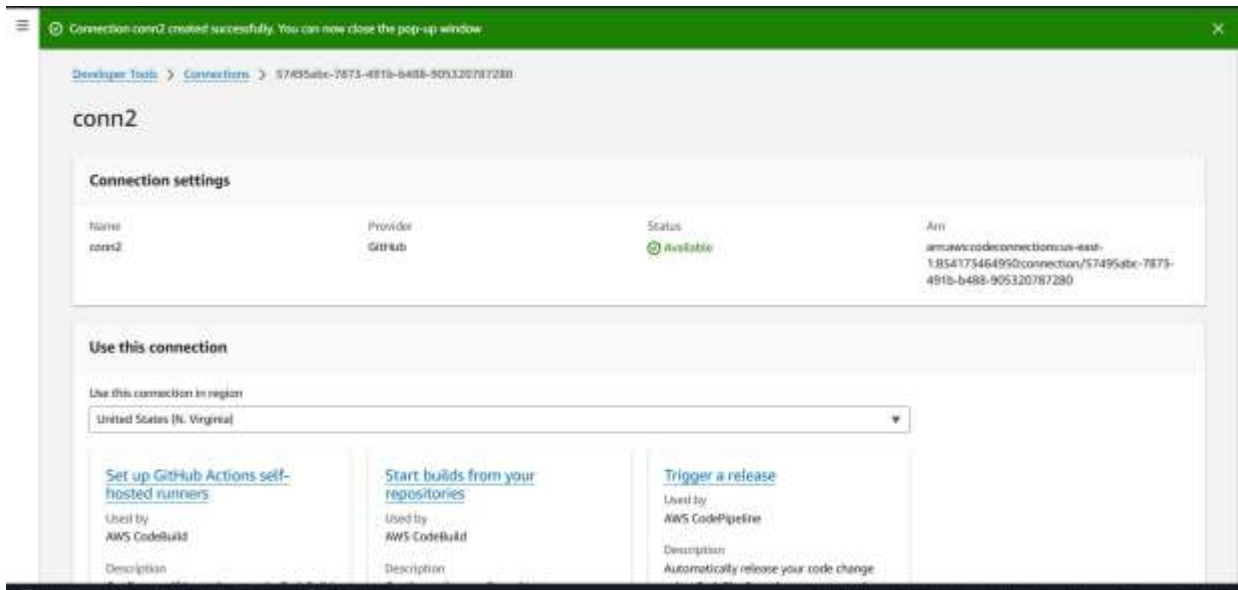


Screenshot 14 — The GitHub authorization page



The connection should eventually display an Available status as seen in screenshot 15.

Screenshot 15 — GitHub Connection Status



Once the connection has been successfully established, the GitHub Source Stage is configured. The repository sprevonix consult is selected as the repository to be monitored by CodePipeline, while main is selected as the source branch. For Output artifact format, CodePipeline default is selected. With this option, the repository files are obtained by CodePipeline and packaged as an artifact that can be passed to the subsequent stages of the pipeline. Next is then selected. At this stage, the connection between GitHub and AWS CodePipeline has been established. Since a separate build process is not required for the static website, the build stage is skipped. The pipeline configuration is then continued to the deployment stage.

The next step is to connect CodePipeline to Amazon S3, On the Add deploy stage page, select: Deploy provider: Amazon S3, Select the S3 bucket created earlier and Click Next.

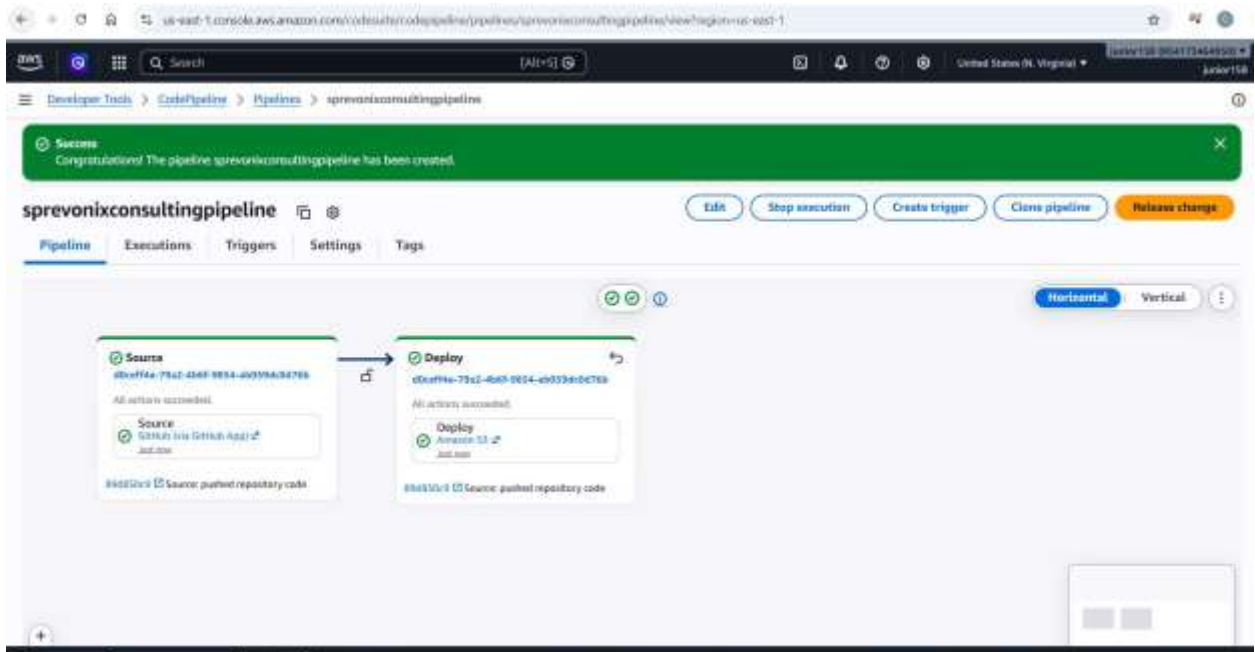
On the Add deploy stage page, Amazon S3 is selected as the Deploy provider. The S3 bucket created earlier for hosting the Sprevonix consulting website is then selected as the deployment destination. The appropriate deployment settings are configured, and Next is selected to continue.

Before the pipeline is created, all configuration settings are reviewed. It is confirmed that the correct GitHub repository and main branch have been selected, the GitHub connection is available, Amazon S3 has been configured as the deployment provider, and the correct S3 bucket has been selected.

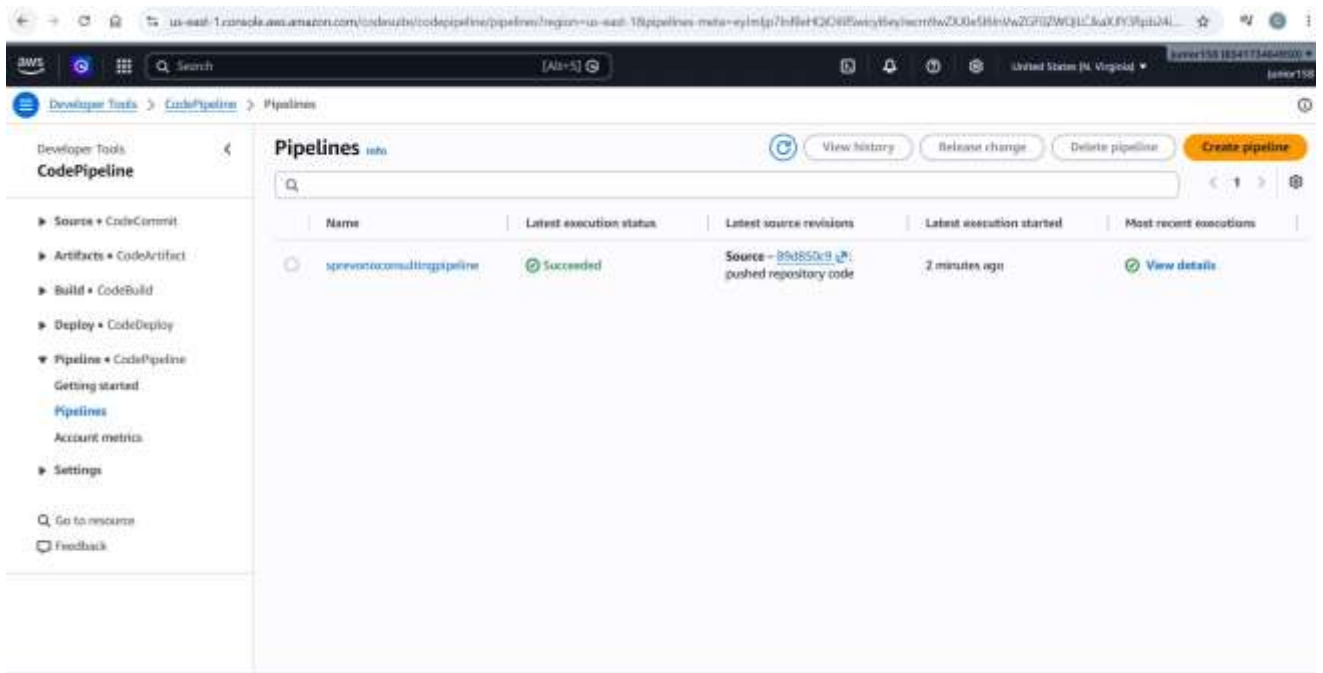
Finally, create pipeline is selected. Once the pipeline has been created, its first execution is automatically initiated. During this execution, the website files are retrieved from the GitHub repository by the source stage and passed to the deploy stage. The application files are then extracted and uploaded to the configured Amazon S3 bucket by the S3 deployment action, as shown in screenshot 16. Then create pipeline is clicked to create the pipeline and the pipeline is created as shown in screenshot 17.

The completed pipeline therefore establishes an automated deployment workflow between GitHub, AWS CodePipeline, and Amazon S3. Whenever changes are committed and pushed to the configured GitHub branch, they can be detected by CodePipeline and deployed to the S3 bucket, reducing the need for manual website file uploads.

Screenshot 16 — Code pipeline deployment page



Screenshot 17 — Code pipeline created.



Reflection

Integrating Amazon S3, GitHub, and AWS CodePipeline

Integrating GitHub, AWS CodePipeline, and Amazon S3 enabled an automated deployment workflow for the Sprevonix consulting website. Each service played a distinct role: GitHub acted as the source code repository, AWS CodePipeline managed and automated the deployment process, and Amazon S3 served as the storage and hosting platform for the static website files. This integration established the following workflow:

GitHub → AWS CodePipeline → Amazon S3 → Website

The process begins with GitHub. Five website files, including `about.html`, `contact.html`, `index.html`, `style.css`, and `main.js`, were uploaded to the GitHub repository as explained earlier. GitHub offers a centralized platform for storing, maintaining, and updating the website source code. When modifications are required, files can be updated and committed to the repository, which removes the need for manual changes in the production environment.

CodePipeline serves as the link between GitHub and Amazon S3. In this project, GitHub was designated as the source provider using a GitHub App connection, which enabled CodePipeline to securely access the repository. The main branch was configured as the source branch, so CodePipeline monitors the repository and retrieves website files from that branch. These files are then packaged as an output artifact for the next stage of the pipeline.

The Sprevonix application is a static website, no separate software compilation or build process was required. As a result, the build stage was omitted, and files were transferred directly from the source stage to the deployment stage. This approach streamlined the pipeline and aligned with the project's architecture.

Amazon S3 serves as the deployment destination. The S3 bucket was preconfigured to store the website's HTML, CSS, and JavaScript files. During deployment, Amazon S3 was selected as the deployment provider, with the existing Sprevonix bucket as the target. When the pipeline executes, CodePipeline retrieves the website files from GitHub, and the S3 deployment action extracts and uploads them to the designated bucket.

Through this implementation, I gained a deeper understanding of integrating cloud services to establish an automated deployment process. GitHub manages the source code, CodePipeline orchestrates the movement and deployment of the code, and S3 stores the deployed website files. Instead of manually transferring files from GitHub to S3 after each update, the pipeline integrates these services into a continuous workflow. This approach simplifies deployment management and provides a robust foundation for future enhancements to the Sprevonix website.

Reflection

Why I used Continuous integration and Continuous deployment CICD.

In the Sprevonix Consulting project, Continuous Integration and Continuous Deployment (CI/CD) was used to make website deployment faster, more reliable, and less manual. Before setting up the CI/CD pipeline, we had to upload website files to the Amazon S3 bucket manually, however, with CI/CD, updates from GitHub to Amazon S3 are automated, making future changes easier to manage. The project documentation shows that CodePipeline was set up to detect changes in the source code and automatically move them through each deployment stage.

The CI/CD approach improved the management of future website updates by automating the transfer of changes from GitHub to Amazon S3. Project documentation indicates that CodePipeline was configured to detect source code changes and automatically progress them through the deployment stages.

Another significant motivation for adopting CI/CD was to designate GitHub as the authoritative source for website files. The Sprevonix website comprises HTML, CSS, and JavaScript files, all initially stored in the GitHub repository. Maintaining these files in GitHub enables proper tracking and management of changes prior to deployment. Rather than modifying files directly in the S3 bucket, updates are committed and pushed to the designated GitHub branch. This approach fosters a more organized development workflow and facilitates understanding of the website's evolution over time.

Implementing CI/CD further illustrated the value of automation in cloud and DevOps engineering. Upon creation, the pipeline's initial execution automatically retrieved website files from GitHub and deployed them to the configured S3 bucket. This automation reduced manual intervention required for website updates. Additionally, it demonstrated how integrating various cloud and development services can establish a unified workflow, rather than managing each service in isolation.

In conclusion, using CI/CD made deploying the Sprevonix website more automated, consistent, and efficient. This experience showed how DevOps practices can cut down on repetitive tasks and create a clear process for making website changes. As the website grows, the pipeline will help deploy updates with less manual work.

Part 5 — Custom Domain Registration Using Amazon Route 53

The purpose of this section is to register a custom domain name using Amazon Route 53. A custom domain provides a professional and easy-to-remember website address, such as `sprevonixconsulting.com`, instead of the default CloudFront domain name. Amazon Route 53 is used for domain registration and DNS management and can be integrated with AWS services such as Amazon CloudFront.

The AWS Management Console is first accessed using the appropriate AWS account credentials. Route 53 is entered in the AWS search bar and selected from the available services. From the Route 53 dashboard, Domains is selected from the navigation pane, followed by Registered domains. The Register domains option is then selected to begin the registration process.

Under Search for domain, the desired domain name, such as `sprevonixconsulting.com`, is entered and searched to confirm its availability. If the domain name is unavailable, an alternative domain name or another supported top-level domain (TLD), such as `.net` or `.org`, can be considered.

Once the desired domain has been confirmed as available, it is selected and added to the registration list. Proceed to checkout is then selected, and the preferred registration period is chosen. The required contact information is provided, which may include the registrant's name, organization, address, telephone number, and email address.

Where supported, privacy protection is enabled so that certain personal contact information is not publicly displayed through domain-registration lookup services. The availability of this feature depends on the selected TLD and its registry requirements.

Before the registration is submitted, the domain name, registration period, contact information, auto-renewal settings, and registration cost are reviewed to ensure that the information is correct. The applicable terms and conditions are accepted, and the registration request is submitted. The registration is then processed by AWS, and the AWS account is charged based on the registration price of the selected domain.

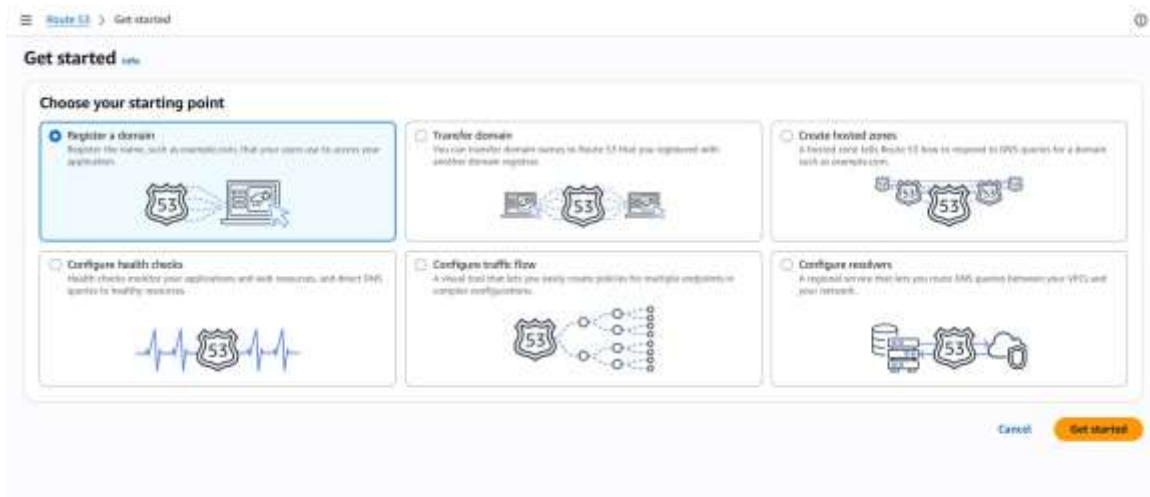
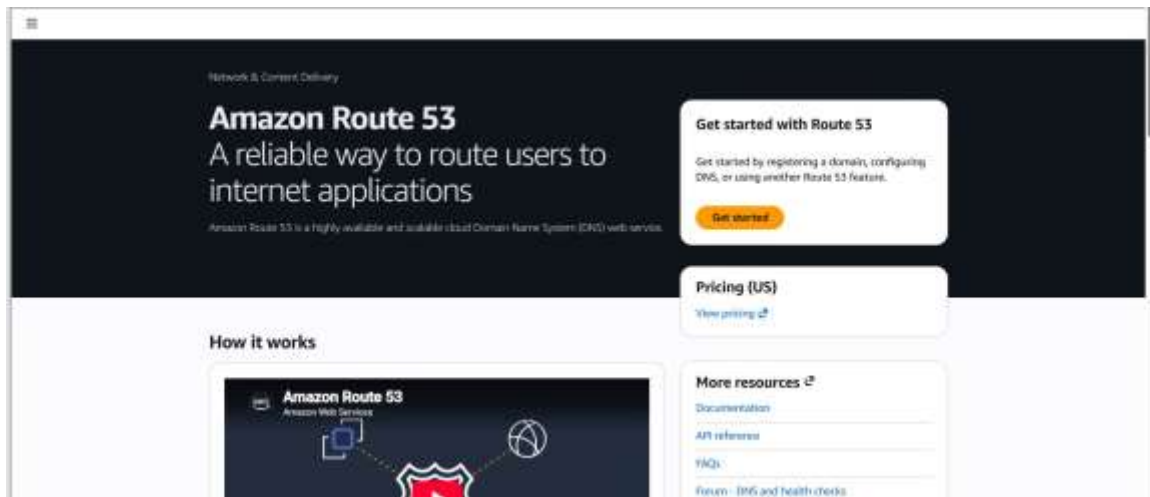
After the registration has been successfully completed, the domain is displayed under Registered domains in Route 53. The domain can then be configured to direct website traffic to the existing CloudFront distribution, allowing the website to be accessed through a professional HTTPS address rather than the default `cloudfront.net` address.

The resulting website architecture is:

User → Route 53 Custom Domain → CloudFront (HTTPS) → Private Amazon S3 Bucket

This configuration provides the Sprevonix Consulting website with a professional domain name while maintaining the secure HTTPS delivery and private S3 configuration established through Amazon CloudFront.

Screenshot 18 — Creation of Amazon Route 53



References

Amazon Web Services. (2024). Hosting a static website using Amazon S3. AWS Documentation.

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/WebsiteHosting.html>

Amazon Web Services. (2024). Getting started with a simple CloudFront distribution. AWS Documentation.

<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/GettingStarted.SimpleDistribution.html>

Amazon Web Services. (2024). Restricting access to an Amazon S3 origin. AWS Documentation.

<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html>

Amazon Web Services. (n.d.). Getting started with AWS CodePipeline. AWS CodePipeline User Guide. [AWS CodePipeline documentation](#)

<https://docs.aws.amazon.com/codepipeline/latest/userguide/welcome-get-started.html>

Amazon Web Services. (n.d.). Registering a new domain with Amazon Route 53. AWS Documentation.

https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/domain-register.html?utm_source=chatgpt.com